Generative AI & LLM Interview Knowledge

LLMs, transformers, prompting, grounding, evaluation and production APIs

Purpose
AI/ML distinctions, LLM fundamentals, attention, embeddings, prompting, structured output, hallucination control and API reliability.

Audience: Final-year engineering students and fresher candidates targeting Python, software development, ML, AI engineering and GenAI roles.

RAG design: Content is explicit and retrieval-friendly so chunks can stand alone. Source identifiers, definitions, examples, troubleshooting and interview reasoning are intentionally separated.

Table of Contents

1. 1. AI, ML, deep learning and GenAI
2. 2. Tokens and context
3. 3. Transformers and attention
4. 4. Embeddings
5. 5. Prompt engineering and structured outputs
6. 6. Temperature and generation controls
7. 7. Hallucinations, grounding and evaluation
8. 8. Production model APIs
9. 9. Choosing an LLM versus ML or deterministic code

1. AI, ML, deep learning and GenAI

AI is the broad field of intelligent systems. ML learns patterns from data. Deep learning uses multi-layer neural networks. Generative AI focuses on generating new content. These categories overlap rather than form a strict sequence.

An LLM is a large neural language model trained on token sequences and adapted for tasks such as instruction following, summarization, coding and question answering.

Term	Interview meaning
AI	Broad intelligent-systems field
ML	Learning patterns from data
Deep learning	Neural-network-based ML
Generative AI	Generation of new content
LLM	Large language model for token sequences

2. Tokens and context

Tokens are tokenizer-specific units that may be words, subwords, punctuation or other fragments. Token counts affect context usage, latency and API cost. Character count is not the same as token count.

A context window is the bounded amount of tokenized information an API/model can consider for a request under its documented limits. More context is not automatically better: irrelevant context can dilute useful evidence.

3. Transformers and attention

Transformers use attention to let token representations incorporate information from other positions. In self-attention, queries, keys and values are used to calculate relevance-weighted combinations.

Multi-head attention lets different heads learn different relationships. Exact architectures vary by model family, so an interview answer should describe the common mechanism without claiming all LLMs are architecturally identical.

```
Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V

Interpretation:
1. Compare query/key representations.
2. Scale scores.
3. Convert scores to weights.
4. Combine value vectors.
```

4. Embeddings

Embeddings map content into numeric vectors useful for similarity or other downstream tasks. Semantic retrieval compares query and document vectors in a shared representation space.

Embedding model identity and dimensions matter. If the embedding model changes, re-indexing is commonly required because vectors from unrelated models should not be mixed casually.

5. Prompt engineering and structured outputs

A useful prompt defines the task, constraints, relevant context, desired format and what to do when evidence is missing. Few-shot examples can demonstrate a pattern but consume context and may introduce unwanted bias.

Structured output is valuable when downstream code needs predictable fields. Prefer schema-constrained features where the provider supports them, and validate any returned data before using it.

```
SYSTEM:
You answer interview questions using supplied context.
```

```
If context is insufficient, say so.
Do not invent source names or citations.

USER:
Explain dependency injection.

CONTEXT:
[FastAPI docs] Dependencies let endpoints declare reusable requirements.
```

6. Temperature and generation controls

Temperature changes sampling behavior in APIs that expose it; lower values often reduce variation but do not guarantee deterministic output. Top-p restricts sampling to a probability mass. Exact controls and ranges are model/API dependent.

Maximum output tokens limits generation length but does not itself guarantee a concise answer. Stop sequences can terminate generation on supported APIs.

Control	General effect	Important caveat
Temperature	More/less sampling variation	Not a truth/factuality knob
Top-p	Probability-mass cutoff	Model/API dependent
Max output	Length cap	Not a quality guarantee
Stop sequence	Ends on pattern	Not universally supported

7. Hallucinations, grounding and evaluation

A hallucination is an unsupported, incorrect or fabricated generated claim. Fluency is not evidence of correctness. Grounding supplies trusted evidence and instructs the model to base important claims on it.

Evaluate retrieval and generation separately. A RAG answer can be wrong because retrieval missed the evidence or because the model misused correct evidence.

- Correctness, groundedness, completeness, relevance, citation accuracy and robustness are useful evaluation dimensions.

8. Production model APIs

Treat model APIs as unreliable network dependencies. Handle authentication errors, rate limits, quotas, timeouts and provider failures. Use bounded retries with backoff only for retryable conditions.

Keep keys server-side in environment variables or a secrets manager. Avoid logging credentials and sensitive prompts unless policy permits it.

```
import time

def call_with_retry(call, attempts=3):
    delay = 1.0
    for attempt in range(attempts):
        try:
            return call()
        except TimeoutError:
            if attempt == attempts - 1:
                raise
            time.sleep(delay)
            delay *= 2
```

9. Choosing an LLM versus ML or deterministic code

Use deterministic code for exact rules such as validation, authorization and arithmetic. Use traditional ML when the problem is a stable prediction target with labeled data. Use an LLM when flexible language understanding/generation is central.

Real systems often combine them: deterministic backend rules for safety, retrieval for evidence, ML for prediction/ranking and an LLM for natural-language interaction.

Need	Good first choice
Exact calculation	Deterministic code
Schema validation	Validation library
Predict label/value	Traditional ML
Semantic search	Embeddings + retrieval
Explain trusted context	LLM + grounding
Authorization	Deterministic backend

Interview question

What is a context window?

Strong answer: The bounded token capacity available to a model/API for the context and generation under that interface. It is not permanent memory.

Follow-up: Why can a larger context still hurt?

Interview question

What causes hallucinations?

Strong answer: Generative models are optimized to produce likely continuations, not to guarantee that every claim is true. Missing context, weak retrieval and ambiguous prompts can increase unsupported output. Grounding and evaluation reduce risk but do not make errors impossible.

Follow-up: How can a RAG system reduce hallucination?

Interview question

Explain attention simply.

Strong answer: Attention lets a token representation weigh information from other token positions according to learned relevance scores. It helps represent relationships across a sequence.

Follow-up: What are queries, keys and values?

Interview question

Why use structured outputs?

Strong answer: Downstream software needs predictable fields and types. A schema makes parsing and validation explicit and failures easier to handle.

Follow-up: What if the provider does not support schema-constrained output?

Interview question

When should you not use an LLM?

Strong answer: When a deterministic rule is simpler and safer, when exact guarantees are required, or when a conventional supervised model is a better fit for a stable prediction problem.

Follow-up: Give an example from backend development.

Scenario: model invents a source

Situation: The application asks the model to write citations but gives it no stable source identifiers.

Reasoning: Pass source IDs/metadata with retrieved chunks and render citations from application data. The model should not be the authority for which files were actually retrieved.

Scenario: API returns intermittent 429/timeout errors

Situation: Treat rate limits and timeouts as dependency failures. Bound concurrency, respect provider limits, retry only retryable failures with backoff, and expose a controlled user-facing fallback.

Reasoning: Retrying every exception forever.

Revision checklist

- Distinguish AI/ML/deep learning/GenAI.

- Explain tokens and context.
- Explain transformer attention conceptually.
- Explain embeddings.
- Write grounded prompts.
- Explain temperature without claiming it guarantees truth.
- Define hallucination and grounding.
- Handle API failures and quotas.
- Protect keys and sensitive data.
- Choose LLM vs ML vs deterministic code.

Reference notes

Primary documentation used for version-sensitive guidance. Verify exact package/API behavior against the versions installed in your project.

https://ai.google.dev/gemini-api/docs

https://ai.google.dev/gemini-api/docs/structured-output

https://ai.google.dev/gemini-api/docs/rate-limits

End of knowledge base document